

Discovering Efficient and Explainable Communication Topologies for LLM-based Multi-Agent Systems via Causal Inference

Junzhi Li^{1,2*}, Peng He^{3*}, Qirui Ji^{2*}, Wei Wang³, Lixiang Liu², Chuxiong Sun^{2†}

¹University of Chinese Academy of Sciences

²National Key Laboratory of Space Integrated Information System, Institute of Software, Chinese Academy of Sciences

³Beijing University of Posts and Telecommunications

lijunzhi25@mails.ucas.ac.cn, {jqirui2022, lixiang, chuxiong2016}@iscas.ac.cn,
{hepeng123, wangwei}@bupt.edu.cn

Abstract

The performance of large language model (LLM)-based multi-agent systems (MAS) largely depends on effective communication topologies. Existing topology generation methods, however, typically learn communication topologies through black-box optimization driven solely by task-level rewards. While effective, such optimization provides little insight into why particular communication edges are selected, making it difficult to identify the critical communication subgraphs responsible for successful collaboration. To address this limitation, we propose E2-Explainer, a model-agnostic framework for providing interpretable explanations of communication topologies produced by arbitrary topology generators. Specifically, we formulate topology explanation as a causal attribution problem that identifies compact communication subgraphs supported by edge-level evidence of task preservation. We obtain this evidence with a Granger-style objective that measures how masking each communication channel changes the task outcome and the stability of the final response. The resulting budgeted subgraphs are then distilled into an amortized explainer, enabling efficient post-hoc explanation without repeated edge-level evaluations at deployment. Extensive experiments on multiple reasoning and coding benchmarks demonstrate that E2-Explainer identifies critical communication subgraphs that preserve successful collaboration. These subgraphs can also be executed directly to prune redundant communication edges, substantially reducing communication costs while maintaining competitive task performance.

Introduction

Large language model (LLM)-based agents have demonstrated remarkable capabilities across a wide range of complex tasks, including question answering (Xu et al. 2024), code generation (Zhang et al. 2024), and autonomous driving (Jin et al. 2023). However, organizing multiple LLM-based agents into a cohesive team capable of exhibiting human-like collective intelligence poses a range of new challenges (Du et al. 2024; Wu et al. 2024; Li et al. 2023; Qian

et al. 2024; Hong et al. 2024). Among these challenges, a fundamental one lies in designing the communication topology—that is, determining how agents should be connected and how they should transmit information during collaboration—as it directly shapes information flow, coordination effectiveness, and ultimately the collective intelligence of the system.

Early studies typically relied on manually specified and predefined topologies (Du et al. 2024; Wu et al. 2024; Li et al. 2023; Qian et al. 2024; Hong et al. 2024). Although simple and effective, these topologies are largely task-agnostic, limiting their adaptability to diverse tasks and evolving collaboration demands. To improve task adaptivity and communication efficiency, recent works have increasingly explored learnable communication topology designers, which broadly fall into two paradigms. Pruning-based approaches begin with an existing collaboration workflow and remove unnecessary agents or communication links through learned gating mechanisms, structured dropout, or importance estimation (Zhang et al. 2025a; Wang et al. 2025; Li et al. 2025). Generation-based approaches, by contrast, formulate topology design as a task-conditioned graph generation problem and synthesize collaboration topologies using autoregressive node-and-edge decoding, mixture-of-experts graph generators, or guided graph diffusion (Li et al. 2026a,b; Jiang et al. 2026; Zhang et al. 2026).

Despite their impressive performance, we observe that the communication topologies generated by existing methods still contain substantial redundancy. As illustrated in Figure 1, removing several communication links in the topologies generated by G-Designer (Zhang et al. 2025b) has little impact on the final task performance and, in some cases, even improves it. This observation suggests that the learned topologies may contain not only redundant edges but also communication patterns that are merely correlated with, rather than causally relevant to, successful collaboration. Such redundancy is difficult to discover under the black-box optimization paradigm adopted by existing methods, where topologies are optimized solely using topology-level task rewards without explicitly attributing the final outcome to individual communication decisions or substructures. Con-

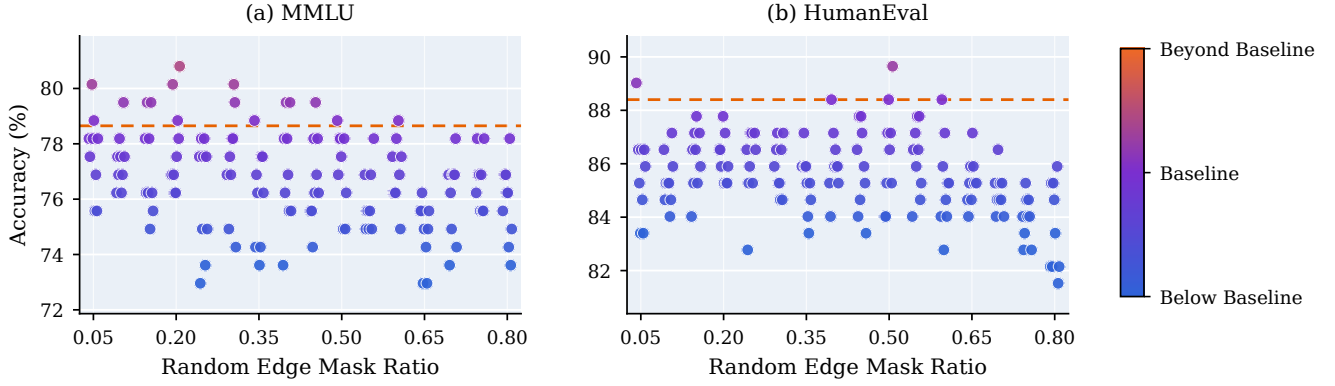


Figure 1: Random edge masking on G-Designer-generated communication graphs for MMLU and HumanEval. The edge masking ratio ranges from 0.05 to 0.80 in increments of 0.05, with ten random seeds evaluated at each ratio. Each point denotes the accuracy obtained with one random mask, while the dashed line marks the performance of the original unmasked G-Designer graph. Most random masks degrade performance, whereas only a few preserve or improve accuracy, indicating that task-preserving compact subgraphs exist but are difficult to identify through random pruning.

sequently, existing topology designers provide little insight into which communication substructures are truly responsible for successful collaboration. This motivates the need for a post-hoc explanation framework that can identify critical causal communication substructures, thereby revealing why a learned topology succeeds and enabling redundant communication edges to be pruned for improved efficiency.

In this work, we explicitly investigate the causal contribution of individual communication edges to the task outcomes achieved by the overall communication topology. Our formulation is inspired by the notion of Granger causality, which considers $x_i \rightarrow Y$ causal if the information provided by x_i improves the prediction of Y (Granger 1969; Wiener 1956). Analogously, in a communication topology, if the information transmitted through an edge improves the cooperative performance of the MAS, that edge can be regarded as causally important to the resulting collaboration. Building upon this intuition, we extend the Granger-style criterion from individual communication edges to local communication substructures, aiming to identify compact causal subgraphs that preserve task-relevant interactions while eliminating redundant communication.

Specifically, we propose E2-Explainer, a model-agnostic post-hoc framework for identifying compact causal subgraphs from communication topologies generated by arbitrary topology designers. Given a communication topology, E2-Explainer estimates the Granger-style causal contribution of each edge by systematically masking it and measuring the resulting change in a task evaluation signal. This signal is primarily defined by the final task outcome and is supplemented with result-level semantic entropy changes to provide denser feedback under sparse rewards. The estimated edge contributions are then integrated with structural constraints to identify compact causal subgraphs that preserve task-relevant communication while eliminating redundant interactions. Since this attribution process requires repeated executions of the underlying LLM-based MAS, we further

train an amortized explainer using the discovered causal subgraphs as supervision. At deployment, the explainer directly predicts a compact causal subgraph from a newly generated topology, avoiding repeated edge-masking evaluations. Extensive experiments across six reasoning and coding benchmarks demonstrate that E2-Explainer consistently improves the performance–communication cost trade-off of multiple topology designers while exhibiting strong transferability across different designers and agent scales.

Our contributions are summarized as follows:

- We formulate post-hoc explanation of optimized LLM-MAS communication graphs as the identification of compact subgraphs that preserve the task behavior of the original topology, and introduce a Granger-style criterion for evaluating the contribution of communication edges.
- We propose E2-Explainer, which distills edge-level causal evidence obtained from offline masking evaluations into budget-specific subgraph supervision and trains an amortized explainer to directly generate compact, collaboration-preserving communication subgraphs without requiring edge-contribution evaluations at test time.
- We conduct extensive experiments across multiple benchmarks, topology optimizers, and agent scales, showing that E2-Explainer reduces communication costs while maintaining competitive task performance and exhibits cross-generator and cross-scale transferability.

Related Work

LLM-based multi-agent systems. LLM-based multi-agent systems organize multiple language-model agents into collaborative workflows, where agents exchange intermediate solutions, critique one another, and aggregate final decisions (Du et al. 2024; Wu et al. 2024; Li et al. 2023; Qian et al. 2024; Hong et al. 2024). They build on decomposed

reasoning, self-consistency, and multi-perspective deliberation to improve LLM reliability (Wei et al. 2022; Wang et al. 2023; Kojima et al. 2022), and have been summarized in recent surveys on agent workflows and applications (Guo et al. 2024; Li et al. 2024; Chen et al. 2024). Our work focuses on their communication structure rather than new roles or prompting strategies.

Communication topology optimization. A growing line of work studies how to design efficient communication topologies for LLM-MAS. Recent methods can be roughly grouped into generative topology methods, which construct task-adaptive collaboration graphs, such as G-Designer and ARG-Designer (Zhang et al. 2025b; Li et al. 2026a), and pruning or selection methods, which remove unnecessary links, agents, or graph components from an existing workflow, such as AgentPrune, AgentDropout, Cut the Crap, and adaptive graph pruning methods (Zhang et al. 2025a; Wang et al. 2025; Li et al. 2025; Cang et al. 2026). These methods show that topology strongly affects the performance–cost trade-off. E2-Explainer is complementary: it treats generated topologies as candidate graphs and learns to refine them into smaller task-preserving communication subgraphs.

Causality in machine learning. Causality provides a principled framework for reasoning about interventions, counterfactuals, and invariant mechanisms (Pearl 2009; Peters, Janzing, and Schölkopf 2017; Schölkopf et al. 2021). In machine learning, causal ideas have been used to analyze model behavior under controlled perturbations and to identify factors that are responsible for predictions (Chattopadhyay et al. 2019; Schwab and Karlen 2019). Granger causality characterizes causal informativeness by whether access to one variable improves the prediction of another (Granger 1969, 1980; Bressler and Seth 2011). In graph learning, this intuition has been connected to explanation methods that identify compact graph structures responsible for preserving or changing model behavior, such as GNNExplainer, PGExplainer, counterfactual explainers, and GEM (Ying et al. 2019; Luo et al. 2020; Lucic et al. 2022; Lin, Lan, and Li 2021). Inspired by these ideas, we adapt Granger-style causal reasoning from differentiable graph models to frozen LLM-MAS communication graphs. Unlike prior graph explanation methods that primarily produce explanations for inspection, E2-Explainer distills offline edge-masking evidence into an amortized explainer that directly outputs budgeted task-preserving communication subgraphs at deployment.

Problem Formulation

LLM-MAS Communication Graphs

We consider an LLM-based multi-agent system with agents $V = \{a_1, \dots, a_n\}$. Given query x , a topology generator $\mathcal{G}_\phi$ produces a directed communication graph

$$G = \mathcal{G}_\phi(x) = (V, E),$$

where $e = (a_i, a_j) \in E$ delivers the output of agent a_i to agent a_j . Executing G yields a final response $y(x, G)$ and a dataset-specific task score $Q(x, G)$, such as exact-match accuracy, execution correctness, or an LLM-judged score.

Post-hoc Communication Explanation

Given a generated graph G , we seek a compact and executable subgraph

$$H = (V_H, E_H), \quad H \subseteq G,$$

that preserves its task behavior. The retained edges explain the communication paths supporting the original collaboration, while executing H may reduce communication overhead. We denote the realized online cost by $C(x, H)$, measured using actual execution tokens. Because removing an edge changes downstream prompts and messages, $C(x, H)$ need not vary monotonically with the number of retained edges or nodes.

Objective and Deployment Constraint

Let $b = (\rho_E, \rho_V)$ denote a subgraph budget, where ρ_E and ρ_V specify the desired edge-pruning and node-pruning ratios, and let $\mathcal{H}_b(G)$ be the corresponding family of compact subgraphs. A valid explanation under budget b should satisfy

$$H_b^* \in \mathcal{H}_b(G), \quad Q(x, H_b^*) \approx Q(x, G). \quad (1)$$

The budget determines compactness, while the task-behavior condition defines faithfulness. We report $C(x, H_b^*)$ as measured efficiency rather than assuming that structural sparsity guarantees a fixed token reduction. Because validating candidate subgraphs requires repeated LLM-MAS executions and may require task feedback, we learn an amortized explainer F_θ :

$$\hat{H}_b = F_\theta(x, G, b), \quad \hat{H}_b \subseteq G. \quad (2)$$

E2-Explainer

As illustrated in Figure 2, E2-Explainer is a model-agnostic post-hoc framework for identifying compact causal subgraphs from communication topologies generated by arbitrary topology designers. Given a communication topology, E2-Explainer estimates the Granger-style causal contribution of each edge by systematically masking it and measuring the resulting change in a task evaluation signal. This signal is primarily derived from the final task outcome and further incorporates topology-level semantic entropy changes to provide denser feedback when the task outcome is sparse or coarse-grained. The estimated causal contributions are then combined with budget constraints and structural requirements to construct executable compact subgraphs that preserve task-relevant communication patterns. Since this attribution process requires repeated edge-masking evaluations of the underlying LLM-based MAS, we use the discovered subgraphs as supervision to train an amortized explainer, which can directly predict a budget-specific subgraph for a newly generated topology at test time.

Edge-Level Granger-Style Causal Attribution

To estimate the Granger-style causal contribution of each communication edge under the original topology, we perform single-edge removal interventions while keeping all other factors fixed. For a query x , graph G , and edge $e = (u, v)$, we preserve the same agents, prompts, decoding configurations, and all other communication channels, while intervening on e by blocking the message transmitted from source

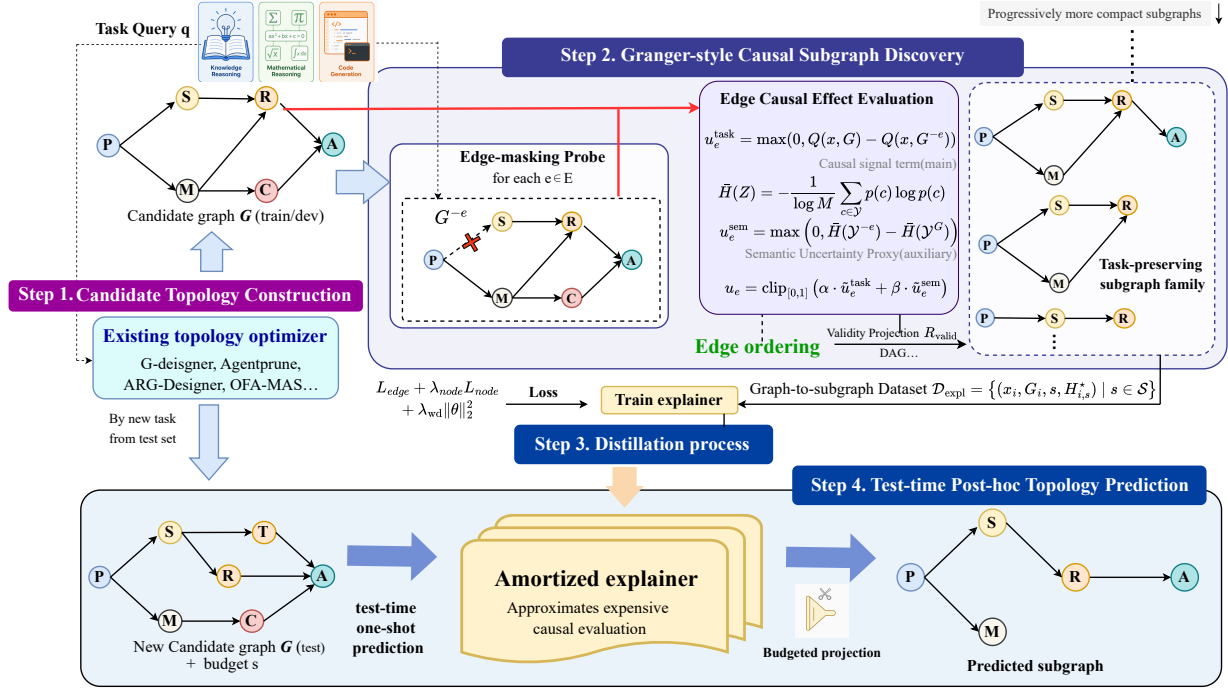


Figure 2: Overview of E2-Explainer. Edge masking estimates preservation utilities from task-score and semantic-entropy changes, which yield budgeted subgraphs for supervision. The amortized explainer then predicts a task-preserving subgraph for an unseen topology in one forward pass.

agent u to target agent v . Following the Granger criterion, we quantify the contribution of e by measuring the change in the evaluation signal before and after this intervention.

Task-level causal contribution. For each active edge $e \in E$ of the generated graph $G = (V, E)$, we construct the intervened graph $G^{-e} = G \setminus \{e\}$ and re-execute the frozen LLM-MAS. Let $Q(x, G)$ and $Q(x, G^{-e})$ denote the task scores obtained before and after intervening on e , respectively. We define the task-level causal contribution of e as

$$\Delta_e^{\text{task}} = Q(x, G) - Q(x, G^{-e}). \quad (3)$$

A positive value indicates that retaining edge e improves the task outcome relative to removing it, whereas a negative value suggests that the information transmitted through e may be detrimental to the task outcome. Since our explanation focuses on communication that supports the original collaboration, we retain only the nonnegative preservation contribution:

$$u_e^{\text{task}} = \max(0, \Delta_e^{\text{task}}). \quad (4)$$

A larger value provides stronger evidence that edge e contributes to preserving the task behavior of the candidate graph, while zero indicates no positive preservation evidence under the observed task metric.

Auxiliary semantic signal. The final task outcome can be too coarse to distinguish edge masks that receive the same task score. We therefore use result-level semantic entropy as

an auxiliary signal to characterize the stability of the final response (Kuhn, Gal, and Farquhar 2023; Farquhar et al. 2024). For the response set $\mathcal{Y}$ obtained from M stochastic executions, we group semantically equivalent responses into classes $\mathcal{C}(\mathcal{Y})$ and compute

$$\bar{H}(\mathcal{Y}) = - \sum_{c \in \mathcal{C}(\mathcal{Y})} p(c) \log p(c), \quad (5)$$

where $p(c)$ is the empirical frequency of class c among the M executions. We use $M = 5$ independently sampled executions for both the original graph and each edge-masked graph. The auxiliary semantic effect of edge e is defined as

$$u_e^{\text{sem}} = \max\left(0, \bar{H}(\mathcal{Y}^{G^{-e}}) - \bar{H}(\mathcal{Y}^G)\right). \quad (6)$$

A positive value indicates that masking e increases the uncertainty of the final response, providing additional evidence that the edge contributes to preserving the behavior of the original graph.

Unified edge effect. We combine the task-level causal effect with the auxiliary semantic stability signal. The resulting edge utility is

$$u_e = \text{clip}_{[0,1]}(\alpha \cdot \tilde{u}_e^{\text{task}} + \beta \cdot \tilde{u}_e^{\text{sem}}), \quad (7)$$

where $\tilde{\cdot}$ denotes normalization within the current graph or calibration batch, and $\alpha, \beta \geq 0$ control the two signals. The

utilities induce a preservation ordering over the edges of G :

$$\pi_G = (e_{(1)}, e_{(2)}, \dots, e_{(|E|)}), \quad u_{e_{(1)}} \geq u_{e_{(2)}} \geq \dots \geq u_{e_{(|E|)}}. \quad (8)$$

This ordering connects edge-level Granger-style attribution to budgeted subgraph construction.

Causal Subgraph Extraction

The edge-level causal contributions obtained above provide a principled basis for extracting compact communication subgraphs. Given a target budget b , E2-Explainer selects edges according to their preservation utilities and constructs a subgraph that retains the most task-relevant information flows while satisfying the structural constraints inherited from the original workflow. This budget-aware extraction process achieves a desirable trade-off between communication sparsity and task preservation without requiring exhaustive subgraph enumeration. Specifically, the budget determines the number of retained edges, while the causal ordering determines which communication paths are preserved.

Budget-Aware Subgraph Selection. The budget $b = (\rho_E, \rho_V)$ specifies the desired retention ratios of edges and optional nodes. For each graph, these ratios are converted into retention counts $k_E(b)$ and $k_V(b)$ according to the graph size. We first select the most important edges according to the preservation ordering obtained from edge attribution:

$$E_b^* = \text{TopK}(\pi_G, k_E(b)). \quad (9)$$

The node budget is subsequently enforced jointly with the structural projection described below to obtain an executable communication subgraph.

Executability-Constrained Projection. To transform the selected high-utility edges into an executable communication topology, we apply an executability-constrained projection:

$$H_b^* = \mathcal{R}_{\text{valid}}(G, E_b^*, k_V(b)). \quad (10)$$

The projection $\mathcal{R}_{\text{valid}}$ only selects edges from the original edge set E and never introduces new links. Therefore, when the original topology G is a directed acyclic graph, the extracted subgraph naturally preserves acyclicity. Specifically, the projection retains mandatory input and final-aggregation agents, removes communication links incident to excluded optional agents, and discards optional agents that no longer participate in valid directed communication paths. When the node budget is active, optional agents are prioritized according to the aggregated preservation utilities of their incident retained edges. The resulting H_b^* therefore satisfies both the specified budget constraints and the execution requirements of the underlying LLM-MAS.

Budget-Specific Explanation Family. Applying Eq. (10) to the budget set $\mathcal{B}$ yields an explanation family

$$\mathcal{F}^*(x, G) = \{H_b^* : b \in \mathcal{B}\}. \quad (11)$$

Each member provides an explanation of the same candidate topology under a different compactness requirement. Collectively, these subgraphs characterize different trade-offs between communication efficiency and task preservation. As the retention budget increases, the extracted subgraphs progressively incorporate lower-ranked communication edges to recover additional information flows.

Amortized Subgraph Explainer

The causal attribution and structural projection procedures described above provide a reliable mechanism for identifying compact, task-preserving subgraphs from arbitrary communication topologies. However, directly applying this procedure to a newly generated topology at test time is computationally expensive, as it requires repeated edge-masking interventions and multiple executions of the underlying LLM-MAS to estimate edge contributions. To overcome this limitation, we amortize the causal extraction process by collecting extracted subgraphs from generated topologies as graph-to-subgraph supervision. Specifically, these causally grounded subgraphs serve as explanation targets that teach an explainer to predict which communication paths should be preserved under different compactness budgets. After training, the amortized explainer can directly generate budget-specific causal subgraphs for unseen topologies without additional causal evaluations.

Graph-to-subgraph supervision. For each query x_i and candidate graph G_i , the extraction procedure produces one target subgraph $H_{i,b}^*$ for every budget $b \in \mathcal{B}$. These targets form the training set

$$\mathcal{D}_{\text{expl}} = \{(x_i, G_i, b, H_{i,b}^*) \mid b \in \mathcal{B}\}. \quad (12)$$

Thus, the explanation family provides supervision rather than an input to the explainer. For a fixed graph, the targets share the same causal preservation ordering but correspond to different pruning budgets. Learning from the complete family therefore teaches the explainer both which communication elements are important and how their retention changes with the requested compactness, allowing one model to serve multiple budgets.

Explainer mapping. We train a parameterized explainer F_θ to approximate the graph-to-subgraph mapping induced by the causal extraction procedure:

$$\hat{H}_{i,b} = F_\theta(x_i, G_i, b), \quad \hat{H}_{i,b} \subseteq G_i. \quad (13)$$

Given a query, a candidate graph, and a pruning budget, F_θ internally predicts edge and node retention probabilities and decodes them into the corresponding budget-specific subgraph. The explainer is independent of the topology generation mechanism, with architectural details deferred to the Appendix.

Training objective. Let $A_{i,b,e}^*$ indicate whether edge e belongs to $H_{i,b}^*$, and let $R_{i,b,v}^*$ indicate whether node v is retained. The explainer predicts the corresponding probabilities $\hat{A}_{i,b,e}$ and $\hat{R}_{i,b,v}$, which are collected into the edge- and node-retention vectors $\hat{\mathbf{A}}_{i,b}$ and $\hat{\mathbf{R}}_{i,b}$, respectively. We optimize

$$\begin{aligned} \mathcal{L}_{\text{edge}} &= \sum_{(x_i, G_i, b, H_{i,b}^*) \in \mathcal{D}_{\text{expl}}} \frac{1}{|E_i|} \sum_{e \in E_i} \text{BCE} \left(A_{i,b,e}^*, \hat{A}_{i,b,e} \right), \\ \mathcal{L}_{\text{node}} &= \sum_{(x_i, G_i, b, H_{i,b}^*) \in \mathcal{D}_{\text{expl}}} \frac{1}{|V_i|} \sum_{v \in V_i} \text{BCE} \left(R_{i,b,v}^*, \hat{R}_{i,b,v} \right), \end{aligned} \quad (14)$$

with the final objective

$$\mathcal{L} = \mathcal{L}_{\text{edge}} + \lambda_{\text{node}} \mathcal{L}_{\text{node}} + \lambda_{\text{wd}} \|\theta\|_2^2. \quad (15)$$

Subgraph prediction. The predicted probabilities are first converted into budget-compatible candidate sets:

$$\begin{aligned} \hat{E}_{i,b} &= \text{TopK} \left(\hat{\mathbf{A}}_{i,b}, k_E(b) \right), \\ \hat{V}_{i,b} &= \text{TopK} \left(\hat{\mathbf{R}}_{i,b}, k_V(b) \right). \end{aligned} \quad (16)$$

Accordingly, the final subgraph is obtained as

$$\hat{H}_{i,b} = F_\theta(x_i, G_i, b) = \mathcal{R}_{\text{valid}} \left(G_i, \hat{E}_{i,b}, \hat{V}_{i,b} \right), \quad (17)$$

where mandatory nodes are retained by $\mathcal{R}_{\text{valid}}$. At deployment, this process requires neither task labels nor repeated edge interventions.

Experiment

Experimental Setup

Datasets and backbone. We evaluate on six benchmarks: AQuA, GSM8K, MultiArith, and SVAMP for mathematical reasoning (Ling et al. 2017; Cobbe et al. 2021; Roy and Roth 2015; Patel, Bhattamishra, and Goyal 2021), MMLU for knowledge-intensive reasoning (Hendrycks et al. 2021), and HumanEval for code generation (Chen et al. 2021). All methods use Qwen3-8B as the backbone LLM (Yang et al. 2025), with Vanilla, CoT (Wei et al. 2022), and SC(CoT) (Wang et al. 2023) as single-agent baselines.

Candidate generators and calibration. We train a single explainer using only G-Designer-generated candidate graphs. For each dataset, 40 calibration inputs are sampled from the training split without overlap with the test split, and edge-masking probes construct graph-to-subgraph supervision. The frozen explainer is then applied without retraining to graphs produced by ARG-Designer, OFA-MAS, AgentPrune, and G-Designer.

Subgraph setting and metrics. The main comparison uses $E25 + N20$ as the default subgraph-size setting, while other settings are examined in the appendix. We report task accuracy and actual online token usage; changes for E2-Explainer are computed relative to the corresponding candidate generator.

Results and Analysis

Main comparison. Table 1 evaluates E2-Explainer as a post-hoc module for four representative topology optimizers. Across all four optimizers, E2-Explainer reduces weighted token usage by 20.1%–25.6%, while preserving or improving their average accuracy. Specifically, it improves the average accuracy of ARG-Designer, OFA-MAS, AgentPrune, and G-Designer by 0.44, 0.53, 0.58, and 1.09 points, respectively. G-Designer+E2-Explainer achieves the highest average accuracy of 90.71 while consuming 25.1% fewer tokens than the original G-Designer. These results directly support the primary objective of E2-Explainer: reducing communication

overhead without sacrificing the collaborative effectiveness of an already optimized topology.

The fine-grained results show that the reduction in communication cost is more consistent than the change in task accuracy. Across the 24 optimizer–dataset combinations, E2-Explainer reduces token usage in every case, with dataset-level reductions ranging from 10.6% to 44.0%. Meanwhile, accuracy improves in 16 combinations and remains unchanged in one. Improvements are particularly consistent on MMLU, AQuA, and HumanEval, where all four topology optimizers benefit from post-processing. Results on GSM8K, MultiArith, and SVAMP are more mixed, indicating that the amount of removable communication depends on both the task and the initial candidate topology. Nevertheless, the universal token reduction and predominantly preserved or improved accuracy show that compact task-preserving subgraphs can often be recovered from optimized communication graphs. The additional accuracy gains further suggest that some candidate topologies still retain redundant or distracting communication whose removal can benefit answer quality.

A key observation is that the improvement transfers beyond the generator used for calibration. The explainer is trained only on G-Designer graphs but applied without retraining to ARG-Designer, OFA-MAS, and AgentPrune. These unseen generators have different construction biases, covering autoregressive graph generation, mixture-of-experts graph generation, and pruning-based topology optimization. Nevertheless, E2-Explainer improves their average accuracy by 0.44–0.58 points and reduces weighted online cost by 20.1%–25.6%. Since these generators are unseen during calibration, the transfer is unlikely to come from memorizing G-Designer-specific adjacency patterns. Instead, the explainer appears to capture reusable cues about which communication paths are necessary under a given task and graph context.

Generalization across agent scales. Table 2 further tests cross-scale generalization by training the explainer on 5-agent G-Designer graphs and directly evaluating it on 6-agent graphs. Without retraining, E2-Explainer improves MMLU accuracy from 79.43 to 81.70 and HumanEval accuracy from 89.26 to 90.08, while also reducing token usage by 10.16% and 19.38%, respectively. These results suggest that the learned explainer transfers beyond the graph size observed during training and can still identify compact task-preserving subgraphs under moderate changes in agent scale.

Component ablation. Table 3 evaluates the roles of the causal and auxiliary semantic-entropy signals. Removing the causal signal leads to a larger degradation, reducing accuracy from 79.74 to 75.82 on MMLU and from 90.69 to 86.88 on HumanEval. The causal-only variant, namely *w/o Sem. Entropy*, performs better than the semantic-entropy-only variant, suggesting that task-level causal effects provide the primary preservation-aware signal for edge valuation. However, causal-only still falls short of the full model. This indicates that the result-level semantic entropy proxy complements causal supervision by densifying the refinement signal when final-task feedback is sparse.

Method	Metric	MMLU	GSM8K	MultiArith	SVAMP	AQuA	HumanEval	Avg.
Vanilla	Acc.	72.78	88.15	96.67	93.40	82.68	81.37	85.84
CoT	Acc.	76.81	91.81	97.67	93.60	83.07	87.58	88.42
SC(CoT)	Acc.	77.34	92.48	97.67	93.83	84.13	87.08	88.76
ARG-Designer	Acc.	76.68	89.18	97.81	93.57	83.95	86.78	88.00
ARG-Designer + E2-Explainer	Acc.	77.99 $\uparrow$ 1.31	86.94 $\downarrow$ 2.24	97.17 $\downarrow$ 0.64	94.67 $\uparrow$ 1.10	84.58 $\uparrow$ 0.63	89.26 $\uparrow$ 2.48	88.44 $\uparrow$ 0.44
	Token Δ	-12.1%	-15.5%	-23.3%	-21.1%	-29.8%	-14.3%	-20.1%
OFA-MAS	Acc.	79.91	91.01	98.21	93.97	84.11	89.26	89.41
OFA-MAS + E2-Explainer	Acc.	81.87 $\uparrow$ 1.96	90.40 $\downarrow$ 0.61	97.41 $\downarrow$ 0.80	92.65 $\downarrow$ 1.32	86.67 $\uparrow$ 2.56	90.62 $\uparrow$ 1.36	89.94 $\uparrow$ 0.53
	Token Δ	-15.1%	-27.2%	-25.5%	-23.8%	-17.8%	-26.8%	-25.6%
AgentPrune	Acc.	78.43	92.78	98.33	94.90	85.04	87.58	89.51
AgentPrune + E2-Explainer	Acc.	79.08 $\uparrow$ 0.65	94.09 $\uparrow$ 1.31	98.33 $\pm$ 0.00	94.20 $\downarrow$ 0.70	86.61 $\uparrow$ 1.57	88.20 $\uparrow$ 0.62	90.09 $\uparrow$ 0.58
	Token Δ	-14.7%	-26.7%	-27.3%	-22.9%	-38.1%	-10.6%	-24.9%
G-Designer	Acc.	78.65	93.05	98.13	95.00	84.52	88.40	89.63
G-Designer + E2-Explainer	Acc.	79.74 $\uparrow$ 1.09	93.77 $\uparrow$ 0.72	98.17 $\uparrow$ 0.04	94.60 $\downarrow$ 0.40	87.30 $\uparrow$ 2.78	90.69 $\uparrow$ 2.29	90.71 $\uparrow$ 1.09
	Token Δ	-31.5%	-44.0%	-28.0%	-20.1%	-37.5%	-19.3%	-25.1%

Table 1: Main results with Qwen3-8B as the backbone LLM. Boldface indicates the best accuracy for each dataset. For E2-Explainer variants, Acc. arrows indicate changes relative to the corresponding candidate generator, while the Avg. column reports average accuracy and weighted token reduction across all datasets.

Setting	MMLU		HumanEval	
	Acc.	Tok. $\downarrow$	Acc.	Tok. $\downarrow$
Original (6-agent)	79.43	–	89.26	–
E2-Explainer (5 $\rightarrow$ 6-agent)	81.70	10.16%	90.08	19.38%

Table 2: Generalization across agent scales on G-Designer-generated communication graphs. The explainer is trained on 5-agent graphs and evaluated on 6-agent graphs. Token reductions are measured against the original 6-agent topology.

Method	MMLU		HumanEval	
	Acc.	Token Δ	Acc.	Token Δ
w/o Causal	75.82	-11.16%	86.88	-5.05%
w/o Sem. Entropy	77.78	-23.42%	87.50	-14.32%
E2-Explainer	79.74	-31.50%	90.69	-19.25%

Table 3: Component ablation of E2-Explainer on G-Designer-generated communication graphs for MMLU and HumanEval.

Effect of Calibration Sources. Table 4 compares explainers trained on G-Designer graphs, AgentPrune graphs, or their mixture and evaluated on both target optimizers. All six calibration–target combinations improve accuracy while reducing token usage, providing evidence of bidirectional cross-optimizer transfer. Source matching remains beneficial: AgentPrune-only calibration achieves the highest accuracy on both datasets for AgentPrune targets, while G-Designer-only performs best on HumanEval for G-

Source	MMLU		HumanEval	
	Acc.	Token Δ	Acc.	Token Δ
<i>Target: G-Designer</i>				
Original	78.65	–	88.40	–
G-Designer	79.74	-31.50%	90.69	-19.25%
AgentPrune	79.08	-14.43%	88.75	-15.40%
Mixed	80.18	-24.55%	89.30	-20.61%
<i>Target: AgentPrune</i>				
Original	78.43	–	87.58	–
G-Designer	79.08	-14.70%	88.20	-10.60%
AgentPrune	81.70	-24.56%	90.07	-19.18%
Mixed	79.08	-26.69%	88.82	-13.43%

Table 4: Effect of calibration sources under $E25 + N20$. *Original* denotes unrefined target graphs, and *Mixed* combines G-Designer and AgentPrune calibration graphs. Boldface marks the best refined accuracy.

Designer targets. Mixed-source calibration obtains the best G-Designer result on MMLU and improves both target optimizers, but does not uniformly outperform single-source calibration. These results suggest that E2-Explainer learns transferable communication patterns while still benefiting from structural characteristics specific to the target optimizer.

Conclusion

We present E2-Explainer, a post-hoc framework for identifying compact and collaboration-preserving communication subgraphs from optimized LLM-MAS topologies. E2-Explainer distills offline edge-level evaluations into an amortized explainer, enabling efficient test-time generation of compact communication subgraphs that preserve collabo-

rative task performance without requiring ground-truth answers or repeated edge-level evaluations. Experiments across multiple topology optimizers show that it reduces communication costs while maintaining task performance, and transfers across graph sources and agent scales.

References

- Bressler, S. L.; and Seth, A. K. 2011. Wiener–Granger Causality: A Well Established Methodology. *NeuroImage*, 58(2): 323–329.
- Cang, Y.; Zhang, X.; Zhao, E.; Ji, Z.; Liu, Y.; He, Y.; Ning, Z.; Chen, Y.; Que, W.; and Shi, L. 2026. Graph-GRPO: Stabilizing Multi-Agent Topology Learning via Group Relative Policy Optimization. In *Findings of the Association for Computational Linguistics: ACL 2026*, 20222–20231. Association for Computational Linguistics.
- Chattopadhyay, A.; Manupriya, P.; Sarkar, A.; and Balasubramanian, V. N. 2019. Neural Network Attributions: A Causal Perspective. In *International Conference on Machine Learning*.
- Chen, M.; et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.
- Chen, W.; Su, Y.; Zuo, J.; Yang, C.; Yuan, C.; Qian, C.; Chan, C.-M.; Qin, Y.; Lu, Y.; Xie, R.; Liu, Z.; and Sun, M. 2024. A Survey on Large Language Model based Autonomous Agents. *Frontiers of Computer Science*, 18(6): 186345.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.
- Du, Y.; Li, S.; Torralba, A.; Tenenbaum, J. B.; and Mordatch, I. 2024. Improving Factuality and Reasoning in Language Models through Multiagent Debate. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*. PMLR.
- Farquhar, S.; Kossen, J.; Kuhn, L.; and Gal, Y. 2024. Detecting Hallucinations in Large Language Models Using Semantic Entropy. *Nature*, 630: 625–630.
- Granger, C. W. J. 1969. Investigating Causal Relations by Econometric Models and Cross-spectral Methods. *Econometrica*, 37(3): 424–438.
- Granger, C. W. J. 1980. Testing for Causality: A Personal Viewpoint. *Journal of Economic Dynamics and Control*, 2: 329–352.
- Guo, T.; Chen, X.; Wang, Y.; Chang, R.; Pei, S.; Chawla, N. V.; Wiest, O.; and Zhang, X. 2024. Large Language Model based Multi-Agents: A Survey of Progress and Challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, 8048–8057.
- Hendrycks, D.; Burns, C.; Basart, S.; Zou, A.; Mazeika, M.; Song, D.; and Steinhardt, J. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*.
- Hong, S.; Zhuge, M.; Chen, J.; Zheng, X.; Cheng, Y.; Zhang, C.; Wang, J.; Wang, Z.; Yau, S. K. S.; Lin, Z.; Zhou, L.; Ran, C.; Xiao, L.; Wu, C.; and Schmidhuber, J. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *International Conference on Learning Representations*.
- Jiang, E. H.; Li, L.; Wan, F.; Liang, X.; Yin, S.; Wu, Y.; Li, X.; Sun, Y.; Wang, W.; Chang, K.-W.; and Wu, Y. N. 2026. Dynamic Generation of Multi LLM Agents Communication Topologies with Graph Diffusion Models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 38042–38060. Association for Computational Linguistics.
- Jin, Y.; Shen, X.; Peng, H.; Liu, X.; Qin, J.; Li, J.; Xie, J.; Gao, P.; Zhou, G.; and Gong, J. 2023. Surrealdriver: Designing generative driver agent simulation framework in urban contexts based on large language model. *arXiv preprint arXiv:2309.13193*, 5(7): 8.
- Kojima, T.; Gu, S. S.; Reid, M.; Matsuo, Y.; and Iwasawa, Y. 2022. Large Language Models are Zero-Shot Reasoners. In *Advances in Neural Information Processing Systems*, volume 35, 22199–22213.
- Kuhn, L.; Gal, Y.; and Farquhar, S. 2023. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. In *International Conference on Learning Representations*.
- Li, B.; Zhao, Z.; Lee, D.-H.; and Wang, G. 2025. Adaptive Graph Pruning for Multi-Agent Communication. *arXiv preprint arXiv:2506.02951*.
- Li, G.; Hammoud, H. A. A. K.; Itani, H.; Khizbullin, D.; and Ghanem, B. 2023. CAMEL: Communicative Agents for “Mind” Exploration of Large Scale Language Model Society. In *Advances in Neural Information Processing Systems*, volume 36.
- Li, S.; Liu, Y.; Wen, Q.; Zhang, C.; and Pan, S. 2026a. Assemble Your Crew: Automatic Multi-Agent Communication Topology Design via Autoregressive Graph Generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 23142–23150.
- Li, S.; Liu, Y.; Zheng, Y.; Li, M.; Nguyen, Q. V. H.; and Pan, S. 2026b. OFA-MAS: One-for-All Multi-Agent System Topology Design based on Mixture-of-Experts Graph Generative Models. In *Proceedings of the ACM Web Conference 2026*, 1333–1344.
- Li, X.; Wang, S.; Zeng, S.; Wu, Y.; and Yang, Y. 2024. A Survey on LLM-based Multi-Agent Systems: Workflow, Infrastructure, and Challenges. *Vicinearth*, 1(1): 9.
- Lin, W.; Lan, H.; and Li, B. 2021. Generative Causal Explanations for Graph Neural Networks. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, 6666–6679. PMLR.
- Ling, W.; Yogatama, D.; Dyer, C.; and Blunsom, P. 2017. Program Induction by Rationale Generation: Learning to Solve and Explain Algebraic Word Problems. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 158–167. Association for Computational Linguistics.

- Lucic, A.; Ter Hoeve, M. A.; Tolomei, G.; de Rijke, M.; and Silvestri, F. 2022. CF-GNNExplainer: Counterfactual Explanations for Graph Neural Networks. In *Proceedings of The 25th International Conference on Artificial Intelligence and Statistics*, volume 151 of *Proceedings of Machine Learning Research*, 4499–4511. PMLR.
- Luo, D.; Cheng, W.; Xu, D.; Yu, W.; Zong, B.; Chen, H.; and Zhang, X. 2020. Parameterized Explainer for Graph Neural Network. In *Advances in Neural Information Processing Systems*, volume 33, 19620–19631.
- Patel, A.; Bhattamishra, S.; and Goyal, N. 2021. Are NLP Models Really Able to Solve Simple Math Word Problems? In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2080–2094. Association for Computational Linguistics.
- Pearl, J. 2009. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2 edition.
- Peters, J.; Janzing, D.; and Schölkopf, B. 2017. *Elements of Causal Inference: Foundations and Learning Algorithms*. MIT Press.
- Qian, C.; Liu, W.; Liu, H.; Chen, N.; Dang, Y.; Li, J.; Yang, C.; Chen, W.; Su, Y.; Cong, X.; Xu, J.; Li, D.; Liu, Z.; and Sun, M. 2024. Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.
- Roy, S.; and Roth, D. 2015. Solving General Arithmetic Word Problems. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, 1743–1752. Association for Computational Linguistics.
- Schölkopf, B.; Locatello, F.; Bauer, S.; Ke, N. R.; Kalchbrenner, N.; Goyal, A.; and Bengio, Y. 2021. Toward Causal Representation Learning. *Proceedings of the IEEE*, 109(5): 612–634.
- Schwab, P.; and Karlen, W. 2019. CXPlain: Causal Explanations for Model Interpretation under Uncertainty. *Advances in Neural Information Processing Systems*, 32.
- Wang, X.; Wei, J.; Schuurmans, D.; Le, Q. V.; Chi, E. H.; Narang, S.; Chowdhery, A.; and Zhou, D. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations*.
- Wang, Z.; Wang, Y.; Liu, X.; Ding, L.; Zhang, M.; Liu, J.; and Zhang, M. 2025. AgentDropout: Dynamic Agent Elimination for Token-Efficient and High-Performance LLM-Based Multi-Agent Collaboration. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 24013–24035. Association for Computational Linguistics.
- Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E. H.; Le, Q. V.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, volume 35, 24824–24837.
- Wiener, N. 1956. The Theory of Prediction. *Modern Mathematics for Engineers*, 165–190.
- Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; Awadallah, A. H.; White, R. W.; Burger, D.; and Wang, C. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *First Conference on Language Modeling*.
- Xu, Y.; He, S.; Chen, J.; Wang, Z.; Song, Y.; Tong, H.; Liu, G.; Zhao, J.; and Liu, K. 2024. Generate-on-Graph: Treat LLM as both Agent and KG for Incomplete Knowledge Graph Question Answering. In Al-Onaizan, Y.; Bansal, M.; and Chen, Y., eds., *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, 18410–18430. Association for Computational Linguistics.
- Yang, A.; Li, A.; Yang, B.; et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*.
- Ying, Z.; Bourgeois, D.; You, J.; Zitnik, M.; and Leskovec, J. 2019. GNNExplainer: Generating Explanations for Graph Neural Networks. In *Advances in Neural Information Processing Systems*, volume 32.
- Zhang, G.; Yue, Y.; Li, Z.; Yun, S.; Wan, G.; Wang, K.; Cheng, D.; Yu, J. X.; and Chen, T. 2025a. Cut the Crap: An Economical Communication Pipeline for LLM-based Multi-Agent Systems. In *International Conference on Learning Representations*.
- Zhang, G.; Yue, Y.; Sun, X.; Wan, G.; Yu, M.; Fang, J.; Wang, K.; Chen, T.; and Cheng, D. 2025b. G-Designer: Architecting Multi-agent Communication Topologies via Graph Neural Networks. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*. PMLR.
- Zhang, K.; Li, J.; Li, G.; Shi, X.; and Jin, Z. 2024. Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 13643–13658.
- Zhang, Z.; Zhou, W.; Li, J.; Fei, H.; Wen, J.; and Ji, W. 2026. RADAR: Redundancy-Aware Diffusion for Multi-Agent Communication Structure Generation. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*. PMLR.

Implementation Details

Explainer Input and Output

The amortized explainer learns the budget-conditioned mapping

$$F_\theta : (x, G, b) \mapsto \hat{H}_b, \quad \hat{H}_b \subseteq G, \quad (18)$$

where x is the task query, $G = (V, E)$ is a candidate communication graph, and $b = (\rho_E, \rho_V)$ specifies the edge- and node-pruning ratios. All inputs are available before executing the predicted subgraph; the explainer does not require ground-truth answers, repeated edge interventions, or internal parameters of the topology generator.

For each edge $e = (u, v) \in E$ and node $v \in V$, the network predicts an edge-retention probability $\hat{A}_{b,e} \in [0, 1]$ and a node-retention probability $\hat{R}_{b,v} \in [0, 1]$. These probabilities are intermediate outputs rather than the final explanation. They are converted into budget-compatible candidate sets and passed through the validity projection to obtain the executable subgraph $\hat{H}_b$.

The explainer therefore learns a budget-conditioned graph-to-subgraph mapping. The same candidate graph can yield different explanations under different pruning budgets, while the supervision is defined by the edge and node membership of the corresponding compact target subgraph H_b^* .

Budget-Conditioned Explainer Architecture

Input representations. Let $\mathbf{q}_x$ denote a fixed embedding of query x , and let $\mathbf{r}_v$ denote a fixed embedding of the role or system prompt assigned to agent v . Each node is also associated with a structural feature vector $\mathbf{t}_v$, containing graph-local information available from the candidate topology, such as normalized in-degree and out-degree and topological position. A shared node encoder produces

$$\mathbf{h}_v = \phi_{\text{node}}([\mathbf{q}_x \| \mathbf{r}_v \| \mathbf{t}_v]), \quad \mathbf{g}_G = \frac{1}{|V|} \sum_{v \in V} \mathbf{h}_v, \quad (19)$$

where $\|$ denotes concatenation and $\mathbf{g}_G$ is a permutation-invariant graph-level summary. The pruning ratios are encoded as

$$\mathbf{z}_b = \phi_b([\rho_E, \rho_V]). \quad (20)$$

Edge and node prediction heads. For an active edge $e = (u, v)$, let $\mathbf{s}_e$ collect edge-local structural features, including the edge type and normalized structural statistics of its endpoints. The shared edge head predicts

$$\hat{A}_{b,e} = \sigma(\phi_{\text{edge}}([\mathbf{h}_u \| \mathbf{h}_v \| \mathbf{g}_G \| \mathbf{z}_b \| \mathbf{s}_e])). \quad (21)$$

The node head predicts

$$\hat{R}_{b,v} = \sigma(\phi_{\text{retain}}([\mathbf{h}_v \| \mathbf{g}_G \| \mathbf{z}_b])). \quad (22)$$

All node and edge instances share the same encoders and prediction heads. Consequently, the number of trainable parameters does not depend on the numbers of agents or communication links, which allows the explainer to process graphs of different sizes.

The modules ϕ_{node} , ϕ_b , ϕ_{edge} , and ϕ_{retain} are implemented as lightweight multilayer perceptrons with parameters shared across graph elements. Fixed task and role representations are computed once and reused during explainer training and inference.

Graph-to-Subgraph Supervision and Decoding

For each calibration pair (x_i, G_i) , causal subgraph construction produces one target $H_{i,b}^*$ for every budget $b \in \mathcal{B}$. The resulting training set is

$$\mathcal{D}_{\text{expl}} = \{(x_i, G_i, b, H_{i,b}^*) \mid b \in \mathcal{B}\}. \quad (23)$$

Each target subgraph defines binary edge- and node-retention labels

$$A_{i,b,e}^* = \mathbb{I}[e \in E(H_{i,b}^*)], \quad R_{i,b,v}^* = \mathbb{I}[v \in V(H_{i,b}^*)]. \quad (24)$$

The explainer is optimized with the edge and node binary-cross-entropy losses defined in the main paper. Because one candidate graph contributes one target for each budget, the model learns both which communication elements should be retained and how the selected subgraph changes with the requested pruning ratios.

At inference time, the predicted probabilities are converted into candidate sets according to the graph-specific retention counts:

$$\hat{E}_b = \text{TopK}(\hat{\mathbf{A}}_b, k_E(b)), \quad \hat{V}_b = \text{TopK}(\hat{\mathbf{R}}_b, k_V(b)). \quad (25)$$

The final explanation is

$$\hat{H}_b = \mathcal{R}_{\text{valid}}(G, \hat{E}_b, \hat{V}_b), \quad (26)$$

where $\mathcal{R}_{\text{valid}}$ enforces the requested node budget together with a minimum-node constraint, removes every edge incident to an excluded node, and retains only feasible directed links between selected agents. The projection ranks feasible elements using the predicted retention scores and selects only nodes and edges already present in G , so it never introduces a new communication link.

Causal Supervision Construction Details

Repeated execution and task score. For every active edge e , we independently execute both the original graph G and the intervened graph G^{-e} for $M = 5$ runs. The task score is the mean outcome over these executions:

$$Q(x, G) = \frac{1}{M} \sum_{m=1}^M Q^{(m)}(x, G). \quad (27)$$

The original graph is re-executed for each evaluated edge rather than sharing one response set across all interventions. All executions use a decoding temperature of 0.2. The original and intervened runs are sampled independently and do not use paired random seeds. The same response sets are used to compute both the mean task outcome and the auxiliary semantic signal.

Result-level semantic signal. Semantic entropy is computed from the final system responses, rather than from the local outputs of the two agents incident to an edge. This choice aligns the auxiliary signal with the system-level quantity whose causal contribution is being estimated. The M final responses are grouped into semantic equivalence classes using the same equivalence procedure for the original and intervened graphs, after which the empirical class entropy is computed as defined in the main paper.

Score normalization. The mean task outcome and its non-negative intervention difference already lie in $[0, 1]$. We therefore retain the task contribution on its original scale and normalize the semantic contribution by the maximum empirical entropy of M samples:

$$\tilde{u}_e^{\text{task}} = u_e^{\text{task}}, \quad \tilde{u}_e^{\text{sem}} = \min\left(1, \frac{u_e^{\text{sem}}}{\log M}\right). \quad (28)$$

The combined preservation utility is then computed with $(\alpha, \beta) = (0.8, 0.2)$.

Pruning counts and validity projection. The budget $b = (\rho_E, \rho_V)$ specifies removal ratios. For a candidate graph $G = (V, E)$, the numbers of removed elements are

$$r_E(b) = \lceil \rho_E |E| \rceil, \quad r_V(b) = \lceil \rho_V |V| \rceil, \quad (29)$$

with retention counts $k_E(b) = |E| - r_E(b)$ and $k_V(b) = |V| - r_V(b)$. Thus, E25 denotes removing 25% of the active edges and N20 denotes removing 20% of the active nodes. After budgeted selection, the validity projection applies the selected node mask, removes incident edges, and retains the highest-ranked feasible links subject to the edge budget and the minimum-node constraint.

Scope of the causal attribution. The intervention procedure estimates a first-order conditional contribution for each edge in the context of the remaining full topology. Following the Granger principle, it measures whether excluding one information channel reduces the mean outcome or the stability of the final response while all other channels remain available. We do not enumerate multi-edge coalitions, whose cost grows combinatorially with graph size. Instead, these conditional edge contributions provide a tractable signal for constructing graph-level supervision, while the end-to-end evaluation of the predicted subgraph measures the aggregate outcome after multiple communication elements are removed.

Implementation Settings

Table 5 summarizes the main settings used for causal supervision construction and explainer training. We train one dataset-specific explainer for each benchmark. The six explainers use the same architecture, optimization configuration, and budget set, but each is trained from the 40 calibration queries of its corresponding dataset. The same frozen Qwen3-8B backbone is used for all methods.

Item	Setting
Semantic samples M	5
Task/semantic weights (α, β)	(0.8, 0.2)
Calibration queries per dataset	40
Training budgets $\mathcal{B}$	E25+N0, E50+N0, E75+N0, E25+N20, E50+N20, E25+N40
Shared hidden dimension	128
Dropout	0.15
Optimizer	AdamW
Learning rate	1×10^{-3}
Weight decay	1×10^{-4}
Batch size / epochs	64 / 200
Backbone LLM	Qwen3-8B
Decoding temperature	0.2
Reported evaluation runs	3
Hardware	4× NVIDIA GeForce RTX 4090 GPUs
Inference framework	PyTorch and vLLM
Token accounting	Online prompt + completion tokens

Table 5: Main implementation settings. Dataset-specific explainers share the same architecture and optimization configuration.

Dataset and Evaluation Details

We evaluate E2-Explainer on six reasoning and code-generation benchmarks using the data scale and evaluation protocol adopted by G-Designer and AgentPrune. For datasets with a dedicated training split, 40 calibration queries are selected from that split and are not used as evaluation queries. For HumanEval, we follow the AgentPrune data split: 40 problems are used for calibration and the remaining 124 problems are used for evaluation. G-Designer, AgentPrune, OFA-MAS, ARG-Designer, and all variants equipped with E2-Explainer are evaluated on exactly the same query set for each benchmark.

Dataset	Task	Calib.	Eval.	Metric
MMLU	Knowledge MCQ	40	153	Accuracy
GSM8K	Math reasoning	40	1,319	Exact match
MultiArith	Math reasoning	40	600	Exact match
SVAMP	Math reasoning	40	1,000	Exact match
AQuA	Algebra MCQ	40	254	Accuracy
HumanEval	Code generation	40	124	Pass@1

Table 6: Dataset statistics and evaluation protocols. HumanEval uses 40 calibration problems and the remaining 124 problems for evaluation. All methods use identical evaluation queries.

MMLU. MMLU evaluates knowledge-intensive reasoning across subjects from STEM, the humanities, and the social sciences. Each query contains a question and four candidate answers. We evaluate on the 153-query validation subset used by G-Designer and AgentPrune and report multiple-choice accuracy.

GSM8K, MultiArith, and SVAMP. These benchmarks evaluate multi-step arithmetic reasoning over natural-

language word problems. Their evaluation sets contain 1,319, 600, and 1,000 queries, respectively. Performance is measured by exact match after extracting and normalizing the final numerical answer.

AQuA. AQuA contains algebraic word problems with five candidate answers. We evaluate on 254 queries and report multiple-choice accuracy based on the final selected option.

HumanEval. HumanEval evaluates function-level Python code generation from a function signature and natural-language specification. Following AgentPrune, 40 problems are used for calibration and the remaining 124 problems are used for evaluation. Pass@1 is computed by executing the generated implementation against the associated unit tests.

Answer extraction and scoring. We follow the answer-processing conventions of the released G-Designer evaluation implementation, which are also used by AgentPrune for the overlapping benchmarks. For MMLU, the evaluator locates the final answer statement and extracts the option label from A–D; AQuA is processed analogously with labels A–E. For GSM8K, MultiArith, and SVAMP, the parser first checks for an explicit final-answer statement or a `\boxed{ }` expression and otherwise uses the last numerical expression in the response. Commas, surrounding spaces, trailing punctuation, and superficial LaTeX formatting are removed before exact comparison with the normalized reference value. For HumanEval, Markdown code fences are removed and the generated Python function is executed against the provided unit tests; a query is counted as correct only when all tests pass within the execution timeout. The same extraction and scoring pipeline is used for every topology generator and its corresponding E2-Explainer variant.

Additional Evaluation Protocols

Controlled candidate-graph comparison. For each query, a topology generator and its E2-Explainer variant start from exactly the same candidate graph and use the same agent

roles, prompts, decoding temperature, and evaluation procedure. The reported accuracy and token results are averaged over three independent runs.

Token accounting. The online communication cost includes the prompt and completion tokens of all retained agent calls and the final aggregation call during execution of the candidate or predicted subgraph. It excludes the offline edge interventions, semantic-entropy sampling, topology-optimizer training, explainer training, and the lightweight explainer forward pass.

Causal ablations. Both ablations reconstruct the target graph-to-subgraph supervision and retrain the explainer. “w/o Causal” uses only the semantic signal by setting $(\alpha, \beta) = (0, 1)$, whereas “w/o Sem. Entropy” uses only the task-level causal contribution by setting $(\alpha, \beta) = (1, 0)$.

Calibration-source analysis. The G-Designer-only and AgentPrune-only variants each use 40 candidate graph instances generated from the corresponding 40 calibration queries. In the mixed setting, the same 40 queries are processed by both topology generators, yielding 80 candidate graph instances in total.

Agent-scale generalization. For MMLU and HumanEval, the scale-generalization experiment trains the explainer on 40 five-agent calibration graphs and directly evaluates it on six-agent graphs generated separately under a six-role G-Designer configuration. The six-agent roles are reconstructed following the role-design logic of G-Designer. The E25+N20 budget and evaluation queries are identical to those used in the main comparison.

Meaning of model-agnostic. In this work, model-agnostic specifically denotes independence from the candidate topology generator. E2-Explainer does not access generator parameters or generator-specific optimization states; it operates on the candidate graph together with the task and agent-role information.

Additional Experimental Results

Results across Different Pruning Budgets

Budget	Metric	MMLU	GSM8K	MultiArith	SVAMP	AQuA	HumanEval	Avg./Total
Original	Acc.	78.65	93.05	98.13	95.00	84.52	88.40	89.63
	Tokens	985,486	1,549,496	3,068,504	11,377,630	1,340,006	725,946	19,047,068
E25+N0	Acc.	78.43↓0.22	93.39↑0.34	98.00↓0.13	94.60↓0.40	87.10↑2.58	88.75↑0.35	90.05↑0.42
	Tokens	985,289↓0.02%	1,472,951↓4.94%	2,919,068↓4.87%	11,325,293↓0.46%	1,315,752↓1.81%	696,908↓4.00%	18,715,261↓1.74%
E50+N0	Acc.	79.08↑0.43	91.95↓1.10	98.17↑0.04	94.30↓0.70	84.52±0.00	90.62↑2.22	89.77↑0.15
	Tokens	972,970↓1.27%	1,496,193↓3.44%	2,906,180↓5.29%	11,300,262↓0.68%	1,305,434↓2.58%	642,172↓11.54%	18,623,211↓2.23%
E75+N0	Acc.	77.78↓0.87	92.71↓0.34	97.83↓0.30	93.80↓1.20	89.29↑4.77	89.38↑0.98	90.13↑0.51
	Tokens	960,849↓2.50%	1,428,945↓7.78%	2,889,917↓5.82%	10,812,162↓4.97%	1,232,270↓8.04%	718,396↓1.04%	18,042,539↓5.27%
E25+N20	Acc.	79.74↑1.09	93.77↑0.72	98.17↑0.04	94.60↓0.40	87.30↑2.78	90.69↑2.29	90.71↑1.09
	Tokens	675,452↓31.46%	867,098↓44.04%	2,208,095↓28.04%	9,091,864↓20.09%	837,906↓37.47%	585,838↓19.30%	14,266,253↓25.10%
E50+N20	Acc.	76.47↓2.18	92.86↓0.19	97.83↓0.30	94.30↓0.70	85.71↑1.19	86.88↓1.52	89.01↓0.62
	Tokens	774,789↓21.38%	1,103,706↓28.77%	2,174,956↓29.12%	9,004,256↓20.86%	889,764↓33.60%	653,860↓9.93%	14,601,331↓23.34%
E25+N40	Acc.	79.64↑0.99	92.55↓0.50	98.00↓0.13	94.10↓0.90	86.11↑1.59	81.88↓6.52	88.71↓0.91
	Tokens	643,030↓34.75%	948,911↓38.76%	1,714,373↓44.13%	6,562,617↓42.32%	575,265↓57.07%	483,916↓33.34%	10,928,112↓42.63%

Table 7: Performance of E2-Explainer on G-Designer candidate graphs under different pruning budgets. Accuracy changes are measured against the unpruned G-Designer graph. Token reductions are reported per dataset, and the final column gives macro-average accuracy and total token usage across all six datasets. E25+N20 is the default setting used in the main paper.

Table 7 provides a more complete view of the accuracy–efficiency trade-off controlled by the edge- and node-pruning budgets. Edge-only pruning largely preserves overall task performance, with the three settings changing average accuracy by only +0.42, +0.15, and +0.51 percentage points. However, even E75+N0 reduces weighted token usage by only 5.27%. This shows that structural sparsity alone does not translate into proportional execution savings when all agents remain active: each retained agent still performs its own reasoning, and the lengths of the remaining messages can change after the topology is modified. At the same time, the gains on AQuA and HumanEval indicate that removing low-contribution channels can suppress redundant or misleading information rather than merely reducing cost.

Introducing node pruning produces substantially larger token reductions, but the benefit depends strongly on the budget. E25+N20 provides the most favorable overall trade-off, improving average accuracy from 89.63 to 90.71 while reducing weighted token usage by 25.10%. It preserves or improves performance on five of the six benchmarks, including gains of 1.09, 2.78, and 2.29 percentage points on MMLU, AQuA, and HumanEval, respectively. Increasing the node-pruning ratio to 40% further reduces token usage by 42.63%, but decreases average accuracy to 88.71, largely because HumanEval drops by 6.52 percentage points. This result suggests that aggressive node removal is particularly risky for code generation, where different roles may provide complementary implementation and verification signals. E50+N20 also removes more edges than E25+N20 but achieves both lower accuracy and a smaller token reduction. The realized cost is therefore jointly determined by node selection, edge

selection, structural projection, and the lengths of the remaining messages, rather than by the nominal sparsity ratio alone.

We perform the complete budget sweep only on G-Designer candidate graphs because its purpose is to characterize budget sensitivity and select one common operating point, rather than tune a different ratio for every topology generator. After selecting E25+N20, we keep the budget fixed when transferring E2-Explainer to AgentPrune, OFA-MAS, and ARG-Designer. This isolates cross-generator generalization from generator-specific budget tuning and evaluates all topology optimizers under the same compression requirement. Selecting a separate budget for each generator on the evaluation set would introduce method-specific tuning advantages and make the comparison less controlled.

Qualitative Case Studies and Execution Traces

We examine four E25N20 examples covering all possible transitions between correct and incorrect predictions. Table 8 summarizes their outcomes and online token usage. In every example, the original candidate graph contains six active spatial masks, whereas E25N20 retains two active spatial masks after removing one agent. The executable acyclic trace contains three same-round communication edges before refinement and one afterward. Active masks describe the selected candidate topology, while the trace records the directed messages that are actually executed after acyclic projection.

The original and E25N20 outputs are independent executions with decoding temperature 0.2. Accordingly, the cases illustrate observed changes in information propagation and

Outcome transition	Dataset / index	Original	E25N20	Online tokens	Token change
Correct → Correct	MultiArith / 419	49	49	6,383 → 4,567	−28.5%
Wrong → Wrong	MMLU / 62	A	A	6,314 → 5,550	−12.1%
Correct → Wrong	MMLU / 151	B	C	11,811 → 10,216	−13.5%
Wrong → Correct	SVAMP / 415	47	3	6,059 → 4,965	−18.1%

Table 8: Overview of the four qualitative cases. Online tokens include the prompt and completion tokens of the agent calls and final aggregation call for the corresponding query. Per-query totals are obtained from the sequential usage records within each evaluation scope.

aggregation under different topologies, rather than paired single-sample proofs that one removed element alone causes the change in correctness.

Case 1: Correct → Correct with lower token usage. The MultiArith query asks how much longer a painter needs to finish 12 rooms when each room requires 7 hours and 5 rooms have already been painted. The reference answer is $(12 - 5) \times 7 = 49$. The original trace is $4tCg \rightarrow 3LK5$, $4tCg \rightarrow 3Ssf$, and $3LK5 \rightarrow 3Ssf$. All four agents return 49. E25N20 removes $4tCg$ (Mathematical Analyst), leaving only $3LK5 \rightarrow 3Ssf$; the three retained agents and the final node still return 49.

The removed Mathematical Analyst is correct, but its derivation duplicates information already available from the other roles and is forwarded to two downstream agents. Removing this node shortens both the number of calls and the prompts received downstream. Total online usage falls from 6,383 to 4,567 tokens, a reduction of 28.5%, while the answer remains correct.

Case 2: Wrong → Wrong under a shared domain misconception. The MMLU query asks which isolated pancreatic-enzyme loss would have the most extensive effect on nutrient absorption in cystic fibrosis. The reference option is C, trypsinogen. Both executions instead select A, lipase. In the original trace, $4MEK \rightarrow 4BNb$, $4MEK \rightarrow 4Cwg$, and $4BNb \rightarrow 4Cwg$ are executed. E25N20 removes $4MEK$ (Knowledgeable Expert) and retains only $4BNb \rightarrow 4Cwg$.

This failure is not primarily caused by one removable noisy message. The active reasoning roles share the same substantive misconception and rank the direct consequences of lipase loss above the broader downstream role of trypsinogen. Pruning reduces token usage from 6,314 to 5,550, but the retained subgraph continues to support the same incorrect answer. This case exposes a limit of topology refinement: removing redundant communication cannot repair an error already shared by the retained agents.

Case 3: Correct → Wrong after removing a useful knowledge source. The MMLU query asks which worker uses a “paddy wagon,” with reference option B, police officer. The original execution returns B. Its realized trace is $4MEK \rightarrow 4BNb$, $4MEK \rightarrow 4Cwg$, and $4BNb \rightarrow 4Cwg$. E25N20 removes $4MEK$ (Knowledgeable Expert), retains only $4BNb \rightarrow 4Cwg$, and returns C, rice farmer.

The removed node provides a useful lexical cue in the original run. After its removal, the only retained communication path propagates an incorrect literal interpretation

from $4BNb$ to $4Cwg$. The execution uses 13.5% fewer tokens, but the connected pair reinforces option C and the final node rejects two independent correct responses. This example shows that a lower-cost topology can lose complementary knowledge and preserve a misleading information path.

Case 4: Wrong → Correct after topology refinement. Figure 3 visualizes the SVAMP example in which Dave initially has 59 files and 15 apps, and retains 30 files and 12 apps after deletion. The requested quantity is the number of deleted apps, so the reference answer is $15 - 12 = 3$. The original G-Designer execution returns 47, whereas E25N20 returns 3.

The original realized trace is $6A7e \rightarrow 8fKH$, $6A7e \rightarrow 7ivC$, and $8fKH \rightarrow 7ivC$. E25N20 removes $8fKH$ (Programming Expert), after which only $6A7e \rightarrow 7ivC$ is executed.

In the original run, the incorrect variable binding is repeated along the connected path and outweighs the only correct response at aggregation. In the independent E25N20 run, all retained agents distinguish files from apps and agree on 3. Token usage also falls from 6,059 to 4,965. The case illustrates how a compact topology can coincide with both lower cost and a more reliable aggregation outcome, while the independent sampling caveat prevents attributing the correction solely to removal of $8fKH$.

Node	Observed input in the realized trace	Logged output summary
<i>Original G-Designer execution</i>		
6bRH (Math Solver)	Query only	Computes 7 remaining rooms and returns $7 \times 7 = 49$.
4tCg (Mathematical Analyst)	Query only	Computes total time $12 \times 7 = 84$, elapsed time $5 \times 7 = 35$, and returns $84 - 35 = 49$.
3LK5 (Programming Expert)	Query and the output of 4tCg	Generates a program that subtracts the painted rooms and returns 49.
3Ssf (Inspector)	Query and the outputs of 4tCg and 3LK5	Verifies the remaining-room calculation and returns 49.
FinalRefer	Final outputs of all four agents	Observes four consistent solutions and returns 49.
<i>E25N20 execution</i>		
6bRH (Math Solver)	Query only	Returns $7 \times 7 = 49$.
3LK5 (Programming Expert)	Query only	Generates an equivalent program and returns 49.
3Ssf (Inspector)	Query and the output of 3LK5	Checks the calculation and returns 49.
FinalRefer	Final outputs of the three retained agents	Receives three consistent answers and returns 49.

Table 9: Execution trace for the correct-to-correct MultiArith case.

Node	Observed input in the realized trace	Logged output summary
<i>Original G-Designer execution</i>		
4MEK (Knowledgeable Expert)	Query only	Produces search terms covering cystic fibrosis, pancreatic function, lipase, trypsinogen, and nutrient absorption, without selecting an option.
6wZy (Critic)	Query only	Notes that trypsinogen activates proteolytic enzymes, but its response ends without a final option.
4BNb (Mathematician)	Query and the output of 4MEK	Prioritizes fat and fat-soluble-vitamin malabsorption and selects A, lipase.
3gkY (Psychologist)	Query only	Uses the same fat-malabsorption argument and selects A.
4Cwg (Historian)	Query and the outputs of 4MEK and 4BNb	Adopts 4BNb’s comparison and selects A.
FinalRefer	Final outputs of all five agents	Returns A, which disagrees with the reference option C.
<i>E25N20 execution</i>		
6wZy (Critic)	Query only	Again discusses the alternatives but does not produce a complete final option.
4BNb (Mathematician)	Query only	Again argues that lipase has the broadest nutritional consequences and selects A.
3gkY (Psychologist)	Query only	Selects A using the same fat-digestion rationale.
4Cwg (Historian)	Query and the output of 4BNb	Repeats the lipase argument and selects A.
FinalRefer	Final outputs of the four retained agents	Returns A again.

Table 10: Execution trace for the wrong-to-wrong MMLU case.

Node	Observed input in the realized trace	Logged output summary
<i>Original G-Designer execution</i>		
4MEK (Knowledgeable Expert)	Query only	Supplies lexical cues linking “paddy wagon” with police officer, while also listing the competing options.
6wZy (Critic)	Query only	Treats the wording as regionally ambiguous but identifies B as the most likely answer.
4BNb (Mathematician)	Query and the output of 4MEK	Compares the police-van and agricultural readings and selects the common police-van meaning, B.
3gkY (Psychologist)	Query only	Independently interprets the term as a police transport vehicle and selects B.
4Cwg (Historian)	Query and the outputs of 4MEK and 4BNb	Follows the common slang interpretation and selects B.
FinalRefer	Final outputs of all five agents	Returns B, matching the reference answer.
<i>E25N20 execution</i>		
6wZy (Critic)	Query only	Still identifies B as the best answer, although with reservations about ambiguity.
4BNb (Mathematician)	Query only	Reinterprets the term literally as a wagon for transporting unmilled rice and selects C.
3gkY (Psychologist)	Query only	Independently retains the police-van interpretation and selects B.
4Cwg (Historian)	Query and the output of 4BNb	Adopts the agricultural interpretation from 4BNb and selects C.
FinalRefer	Final outputs of the four retained agents	Chooses C despite two independent agents selecting B.

Table 11: Execution trace for the correct-to-wrong MMLU case.

Case Study: E25N20 Topology Refinement on SVAMP

Dave had 59 files and 15 apps on his phone. After deleting some apps and files, he had 12 apps and 30 files left. How many apps did he delete?

Ground-truth answer: 3

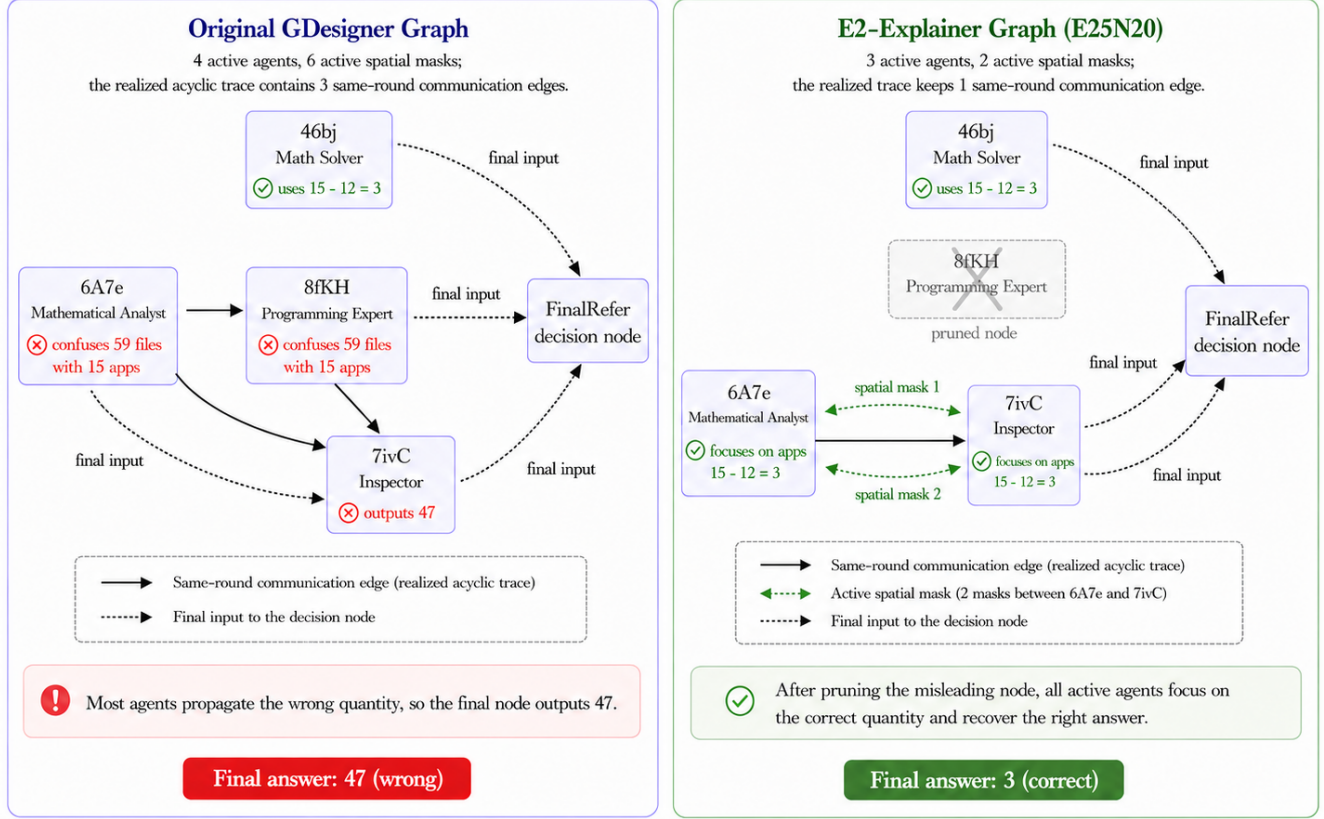


Figure 3: Wrong-to-correct case on SVAMP index 415. The original graph contains four active agents and six active spatial masks, and its realized acyclic trace contains three same-round communication edges. E25N20 removes the Programming Expert, retains three agents and two active spatial masks, and realizes one same-round communication edge.

Node	Observed input in the realized trace	Logged output summary
<i>Original G-Designer execution</i>		
46bj (Math Solver)	Query only	Correctly uses the app counts and returns $15 - 12 = 3$.
6A7e (Mathematical Analyst)	Query only	Misbinds 59 files as the original number of apps and returns $59 - 12 = 47$.
8fKH (Programming Expert)	Query and the output of 6A7e	Sets <code>original_apps=59</code> in its generated program and returns 47.
7ivC (Inspector)	Query and the outputs of 6A7e and 8fKH	Accepts the same variable assignment and returns 47.
FinalRefer	Final outputs of all four agents	Selects 47 from three mutually consistent but incorrect responses and rejects the isolated answer 3.
<i>E25N20 execution</i>		
46bj (Math Solver)	Query only	Returns $15 - 12 = 3$.
6A7e (Mathematical Analyst)	Query only	Correctly binds 59 and 30 to files and 15 and 12 to apps, then returns 3.
7ivC (Inspector)	Query and the output of 6A7e	Verifies the app-based subtraction and returns 3.
FinalRefer	Final outputs of the three retained agents	Receives three consistent answers of 3 and returns 3.

Table 12: Execution trace for the wrong-to-correct SVAMP case in Figure 3.

Generalization to Hand-Crafted Communication Topologies

We further examine whether the frozen explainer trained only on G-Designer-generated graphs can transfer to manually specified communication structures. We consider five rep-

resentative topology families, including complete, random, layered, chain, and star graphs. E2-Explainer is directly applied to these graphs without additional causal supervision or retraining.

Method	Metric	MMLU	GSM8K	MultiArith	SVAMP	AQuA	HumanEval
Complete	Acc.	77.39	90.71	97.50	94.31	83.90	87.09
Complete + E2-Explainer	Acc.	78.43 $\uparrow$ 1.04	91.35 $\uparrow$ 0.64	97.67 $\uparrow$ 0.17	94.50 $\uparrow$ 0.19	84.64 $\uparrow$ 0.74	88.70 $\uparrow$ 1.61
	Token Δ	-17.4%	-36.5%	-30.9%	-22.4%	-28.9%	-25.1%
Random	Acc.	76.78	91.01	97.17	93.97	84.70	86.88
Random + E2-Explainer	Acc.	77.45 $\uparrow$ 0.67	90.97 $\downarrow$ 0.04	97.33 $\uparrow$ 0.16	94.09 $\uparrow$ 0.12	85.03 $\uparrow$ 0.33	87.09 $\uparrow$ 0.21
	Token Δ	-18.7%	-21.2%	-19.7%	-15.6%	-17.6%	-14.3%
Layer	Acc.	77.08	91.24	96.83	94.40	85.49	88.38
Layer + E2-Explainer	Acc.	78.10 $\uparrow$ 1.02	91.28 $\uparrow$ 0.04	97.41 $\uparrow$ 0.58	94.91 $\uparrow$ 0.51	85.43 $\downarrow$ 0.06	89.11 $\uparrow$ 0.73
	Token Δ	-23.6%	-30.1%	-27.4%	-22.8%	-33.9%	-19.9%
Chain	Acc.	78.05	91.09	96.67	94.62	83.51	86.29
Chain + E2-Explainer	Acc.	78.37 $\uparrow$ 0.32	90.67 $\downarrow$ 0.42	96.67 $\pm$ 0.00	94.95 $\uparrow$ 0.33	83.46 $\downarrow$ 0.05	85.88 $\downarrow$ 0.41
	Token Δ	-15.2%	-25.0%	-24.1%	-14.3%	-15.4%	-7.93%
Star	Acc.	77.08	90.78	97.67	93.85	83.90	83.25
Star + E2-Explainer	Acc.	78.03 $\uparrow$ 0.95	91.43 $\uparrow$ 0.65	97.83 $\uparrow$ 0.16	94.17 $\uparrow$ 0.32	85.82 $\uparrow$ 1.92	86.13 $\uparrow$ 2.88
	Token Δ	-29.6%	-35.4%	-22.8%	-17.9%	-32.5%	-20.1%

Table 13: Generalization of E2-Explainer to manually specified communication topologies. The explainer is trained only on G-Designer-generated graphs and applied without retraining. Accuracy arrows indicate changes relative to the corresponding unrefined topology.

Table 13 shows that E2-Explainer transfers effectively beyond the generated graph distribution used for training. It improves or matches accuracy in 26 of the 30 dataset-topology combinations while reducing token usage across every topology and benchmark. The strongest gains appear on the star topology, including improvements of 1.92 points on AQuA and 2.88 points on HumanEval. The layered topology also benefits consistently, reaching 89.11 on HumanEval while reducing token usage by 19.9%. Even for the chain topology, whose sparse structure leaves less removable redundancy, the explainer largely preserves task performance and still reduces token usage on all six benchmarks.

These results are notable because the explainer is calibrated exclusively on G-Designer graphs but is directly applied to fixed topologies with substantially different structural patterns. Its improvements therefore cannot be attributed solely to memorizing G-Designer-specific adjacency structures. Instead, the explainer captures transferable task- and structure-dependent cues for retaining useful communication elements.

Additional Redundancy Analysis on OFA-MAS

To examine whether the redundancy observed on G-Designer is specific to a particular topology designer, we repeat the same random edge-masking analysis on OFA-MAS, as shown in Figure 4. Similar to the observations on G-Designer, most random edge masks degrade performance on both MMLU and HumanEval, while only a small fraction

of them preserve or slightly improve the original accuracy. This pattern suggests that OFA-MAS-generated topologies also contain redundant communication links and that task-preserving compact subgraphs do exist.

At the same time, the rarity of these successful random masks shows that such subgraphs are difficult to identify through unguided pruning alone. In other words, redundancy does not imply that arbitrary pruning is safe. This further motivates the need for a task-conditioned explainer that selects compact communication subgraphs according to causal contribution rather than random deletion.

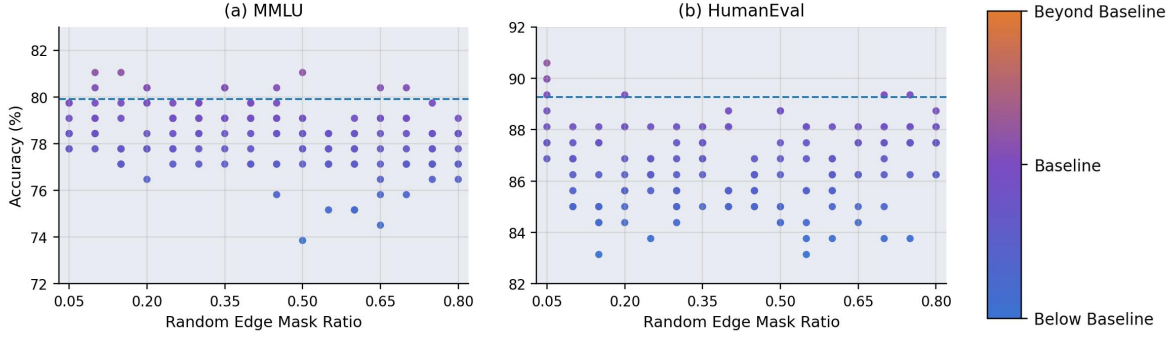


Figure 4: Random edge masking on OFA-MAS-generated communication graphs for MMLU and HumanEval. The edge masking ratio ranges from 0.05 to 0.80 in increments of 0.05. Each point denotes the accuracy obtained under one retained random mask, while the dashed line marks the performance of the original unmasked OFA-MAS graph. For visualization, one extreme run is omitted at each masking ratio. The resulting distribution shows that although a few random masks preserve or even improve the original performance, most random masks degrade accuracy, indicating that task-preserving compact subgraphs exist but are difficult to identify through unguided pruning.

Algorithmic Details

Causal Supervision Construction and Explainer Prediction

Algorithms 1 and 2 summarize the offline construction of graph-to-subgraph supervision and the training and test-time prediction procedures, respectively.

Algorithm 1: Causally Grounded Supervision Construction

Require: $\mathcal{D}_{\text{cal}}, \mathcal{G}_\phi, \mathcal{B}, M, \alpha, \beta$
Ensure: Graph-to-subgraph dataset $\mathcal{D}_{\text{expl}}$

- 1: $\mathcal{D}_{\text{expl}} \leftarrow \emptyset$
- 2: **for** each $x \in \mathcal{D}_{\text{cal}}$ **do**
- 3: $G = (V, E) \leftarrow \mathcal{G}_\phi(x)$
- 4: **for** each active edge $e \in E$ **do**
- 5: $(Q_G^e, \mathcal{Y}^{G,e}) \leftarrow \text{EVALUATE}(x, G, M)$
- 6: $G^{-e} \leftarrow \text{INTERVENE}(G, e)$
- 7: $(Q_e, \mathcal{Y}^{G^{-e}}) \leftarrow \text{EVALUATE}(x, G^{-e}, M)$
- 8: $u_e^{\text{task}} \leftarrow \max(0, Q_G^e - Q_e)$
- 9: $u_e^{\text{sem}} \leftarrow \max(0, \tilde{H}(\mathcal{Y}^{G^{-e}}) - \tilde{H}(\mathcal{Y}^{G,e}))$
- 10: $\tilde{u}_e^{\text{task}} \leftarrow u_e^{\text{task}}$
- 11: $\tilde{u}_e^{\text{sem}} \leftarrow \min(1, u_e^{\text{sem}} / \log M)$
- 12: $u_e \leftarrow \text{clip}_{[0,1]}(\alpha \tilde{u}_e^{\text{task}} + \beta \tilde{u}_e^{\text{sem}})$
- 13: **end for**
- 14: $\pi_G \leftarrow \text{SORTDESCENDING}(E, \{u_e\}_{e \in E})$
- 15: **for** each budget $b \in \mathcal{B}$ **do**
- 16: $(k_E, k_V) \leftarrow \text{RETENTIONCOUNTS}(G, b)$
- 17: $E_b^* \leftarrow \text{TOPK}(\pi_G, k_E)$
- 18: $H_b^* \leftarrow \mathcal{R}_{\text{valid}}(G, E_b^*, k_V)$
- 19: $\mathcal{D}_{\text{expl}} \leftarrow \mathcal{D}_{\text{expl}} \cup \{(x, G, b, H_b^*)\}$
- 20: **end for**
- 21: **end for**
- 22: **return** $\mathcal{D}_{\text{expl}}$

Algorithm 2: Explainer Training and Test-Time Prediction

Require: $\mathcal{D}_{\text{expl}}, \lambda_{\text{node}}, \lambda_{\text{wd}}$
Ensure: Trained explainer F_θ

- 1: Initialize parameters θ
- Training**
- 2: **for** each minibatch $\mathcal{S} \subseteq \mathcal{D}_{\text{expl}}$ **do**
- 3: **for** each $(x_i, G_i, b, H_{i,b}^*) \in \mathcal{S}$ **do**
- 4: $(\mathbf{A}_{i,b}^*, \mathbf{R}_{i,b}^*) \leftarrow \text{INDICATORS}(G_i, H_{i,b}^*)$
- 5: $(\hat{\mathbf{A}}_{i,b}, \hat{\mathbf{R}}_{i,b}) \leftarrow F_\theta(x_i, G_i, b)$
- 6: **end for**
- 7: Compute $\mathcal{L}_{\text{edge}}$ and $\mathcal{L}_{\text{node}}$ on $\mathcal{S}$
- 8: $\mathcal{L} \leftarrow \mathcal{L}_{\text{edge}} + \lambda_{\text{node}} \mathcal{L}_{\text{node}} + \lambda_{\text{wd}} \|\theta\|_2^2$
- 9: Update θ by minimizing $\mathcal{L}$
- 10: **end for**
- Test-time prediction**
- 11: Receive query x , graph G , and budget b
- 12: $(\hat{\mathbf{A}}_b, \hat{\mathbf{R}}_b) \leftarrow F_\theta(x, G, b)$
- 13: $(k_E, k_V) \leftarrow \text{RETENTIONCOUNTS}(G, b)$
- 14: $\hat{E}_b \leftarrow \text{TOPK}(\hat{\mathbf{A}}_b, k_E)$
- 15: $\hat{V}_b \leftarrow \text{TOPK}(\hat{\mathbf{R}}_b, k_V)$
- 16: $\hat{H}_b \leftarrow \mathcal{R}_{\text{valid}}(G, \hat{E}_b, \hat{V}_b)$
- 17: **return** $\hat{H}_b$
